Cross-Site Request Forgery

CSRF — Complete Security Reference Guide

1. What Is CSRF?

Cross-Site Request Forgery (CSRF) is a web security vulnerability that tricks an authenticated user into unknowingly submitting a malicious request to a web application they are currently logged into. The server cannot distinguish the forged request from a legitimate one because it carries the victim's real session cookie.

The key insight: **browsers automatically attach cookies to every request** sent to a domain — regardless of which website triggered that request. An attacker on evil.com can cause your browser to fire a request to bank.com, and the browser will dutifully include your bank.com session cookie.

The 3 Conditions That Must ALL Be True for CSRF to Work
1. A relevant action exists — something worth forging (e.g., change email, transfer money, delete account)
2. Cookie-only authentication — the app relies solely on session cookies to identify the user
3. No unpredictable request parameters — all parameters can be determined or guessed by the attacker

2. How CSRF Works — Step-by-Step Attack Flow

The attack exploits the browser's automatic cookie-sending behaviour. Here is exactly how a CSRF attack unfolds from start to finish:

Step 1 — Victim logs in

The victim visits bank.com and authenticates. The server issues a session cookie.

The browser stores it: Cookie: session=abc123xyz

Step 2 — Attacker prepares a forged page

The attacker creates a page on evil.com containing a hidden HTML form or

an auto-submitting script that targets bank.com/transfer.

Step 3 — Victim visits the malicious page

The attacker tricks the victim into visiting evil.com — via a phishing email,

a chat message, a social media post, or an embedded iframe.

Step 4 — Browser fires the forged request

The malicious page triggers a POST to bank.com/transfer.

The browser automatically attaches the session cookie to the request.

Step 5 — Server trusts the request

bank.com receives a valid-looking request with a legitimate session cookie.

It processes the transfer as if the victim intentionally sent it.

Step 6 — Damage done

The money is transferred, the email is changed, or the account is deleted —

without the victim ever knowingly authorising the action.

3. Concrete Example — Changing a Victim's Email

Suppose a web app lets users change their email with this request:

Legitimate Request (sent by the victim from the real app):

```
POST /account/change-email HTTP/1.1
Host: vulnerable-app.com
Cookie: session=victim_session_token
Content-Type: application/x-www-form-urlencoded

email=victim@gmail.com&submit;=1
```

Attacker's Malicious HTML Page (on evil.com):

```
<html>
<body onload="document.forms[0].submit()">
<form action="https://vulnerable-app.com/account/change-email"
method="POST">
<input type="hidden" name="email" value="attacker@evil.com">
<input type="hidden" name="submit" value="1">
</form>
</body>
</html>
```

When the victim visits this page while logged into vulnerable-app.com, the form auto-submits. The browser sends the POST with the victim's real session cookie. The server changes the email to attacker@evil.com. The attacker can then use 'Forgot password' to take over the account entirely.

4. What Can an Attacker Do With CSRF?

Action Type	Example Impact
Account takeover	Change email/password → use Forgot Password → full control
Financial fraud	Transfer money, make purchases, change payment details
Data destruction	Delete files, wipe account data, remove posts
Privilege escalation	Add attacker as admin, change user roles
Credential harvesting	Change email to attacker-controlled address
Social engineering	Post content as victim, send messages as victim
Infrastructure abuse	On internal admin panels: create users, change configs

5. Defences Against CSRF

5.1 CSRF Tokens (Primary Defence)

A CSRF token is a secret, unpredictable, per-session (or per-request) value that the server embeds in every form or API response. The attacker on evil.com cannot read this value because of the Same-Origin Policy, so they cannot include it in their forged request. The server rejects any request that is missing or has an incorrect token.

How it looks in an HTML form:

```
<form action="/account/change-email" method="POST">
<input type="hidden" name="csrf"
value="xK9mP2qR7vN4wL1jT8uA3cE6">
<input type="email" name="email">
<button type="submit">Change Email</button>
</form>
```

Requirements for a valid CSRF token: unpredictable (cryptographically random), tied to the user's session, validated server-side on every state-changing request, and transmitted outside of cookies (in the form body or a custom header).

5.2 SameSite Cookie Attribute

The SameSite attribute on a cookie tells the browser when to include that cookie in cross-site requests. It is a powerful defence but has important nuances.

Value	Behaviour	CSRF Protection
Strict	Cookie NEVER sent on any cross-site request	Full — but can break legitimate cross-site flows (e.g., OAut
Lax	Cookie sent on top-level GET navigations only (e.g., clicking a link). NOT sent on cross-site POST, iframe, or resource loads	Partial — protects POST-based CSRF; does NOT protect G
None	Cookie always sent cross-site (must also set Secure flag)	No protection — same as old browser behaviour

Important: Chrome now defaults new cookies to SameSite=Lax if no attribute is specified. However, this is not a substitute for explicit CSRF tokens — browser behaviour varies, older browsers ignore SameSite, and Lax still allows GET-based attacks.

5.3 Referer / Origin Header Validation

The server can check the Origin or Referer header to confirm the request originated from the same site. If the header is missing or points to an external domain, the server rejects the request. This is a secondary defence — headers can sometimes be stripped by proxies or privacy tools, so this should not be the sole protection.

5.4 Double Submit Cookie Pattern

A CSRF token is set as a cookie AND also required as a request parameter. The server checks that both values match. Since an attacker cannot read the cookie value (cross-origin), they cannot replicate it in the body. Useful for stateless APIs where server-side session storage is not available.

6. Common CSRF Defence Bypasses

CSRF protections are frequently misconfigured. These are the most common ways defences fail in practice — relevant for both attackers testing and developers hardening:

Token not tied to the session

The server validates that a token exists and is valid globally, but doesn't check that it belongs to THIS session. An attacker can use their own valid token in a forged request targeting the victim.

Token validated only when present

If the token is present it's checked — but if the attacker simply REMOVES the token parameter entirely, the server skips validation and processes the request.

Token stored in a cookie

If the CSRF token is placed in a cookie, the attacker can exploit a subdomain XSS or header injection to set the cookie and then include the matching value in the body.

Method switching (POST to GET)

Many apps protect POST endpoints but forget to protect the same action via GET. If the server accepts GET /account/change-email?email=attacker@evil.com, SameSite=Lax won't help because Lax permits GET navigations cross-site.

SameSite bypass via sibling subdomain

SameSite is site-scoped, not origin-scoped. If app.example.com is the target and attacker controls staging.example.com (same site, different origin), SameSite cookies will still be sent in requests from the attacker's subdomain.

Referer header strippable

Some proxy configs, browser privacy settings, or meta tags (Referrer-Policy: no-referrer) strip the Referer header entirely. If the server allows missing Referer headers, an attacker can strip it and bypass Referer-based validation.

7. CSRF vs XSS — Key Differences

	CSRF	XSS
What it abuses	Browser's automatic cookie sending	Trust the server places in user input
Where attacker is	External site (evil.com)	Injected into the target site itself
Victim interaction	Visit a malicious page	Visit the target page (with payload)
Reads response?	No — blind attack	Yes — can read any page content
CSRF token stops it?	Yes	No — XSS can steal the CSRF token too
SameSite stops it?	Partially (Lax/Strict)	No
Severity	High — state-changing actions	Critical — full script execution

Note: XSS defeats CSRF tokens. If an attacker can run JavaScript on the target origin, they can read the CSRF token from the DOM and include it in their forged request — making CSRF tokens useless in the presence of XSS. This is why XSS is rated more critically: it bypasses most other client-side protections.

8. Testing for CSRF Using Burp Suite

Identify state-changing requests

Browse the application and look for requests that modify data: POST /change-email, POST /transfer, POST /delete-account. These are CSRF candidates.

Send to Repeater

Right-click the request in Proxy history → Send to Repeater.

Check if removing or changing the CSRF token still results in a successful response.

Use 'Generate CSRF PoC'

Right-click any request → Engagement tools → Generate CSRF PoC.

Burp auto-generates an HTML page that replicates the request.

Open it in a browser while logged in to test if the action executes.

Check SameSite attribute

In the response that sets the session cookie, inspect the Set-Cookie header.

If SameSite is absent, None, or Lax — check for GET-based equivalents of the action.

Test token removal

Remove the csrf parameter entirely and resend. If the server accepts it: vulnerable.

Also try: empty value, a token from a different session, or changing one character.

Test Referer stripping

Remove the Referer header in Repeater and resend.

If accepted: the server doesn't enforce origin checking.

9. Quick Reference — CSRF at a Glance

Topic	Key Point
Full name	Cross-Site Request Forgery
Also called	XSRF, Sea-Surf, Session Riding, One-Click Attack
OWASP rank	Was #8 in OWASP Top 10 2017; merged into Broken Access Control category in 2021
Root cause	Browser automatically sends cookies with every request to a domain
Primary defence	CSRF tokens — unpredictable, session-tied, validated server-side
Secondary defence	SameSite=Strict or Lax cookie attribute
Tertiary defence	Origin/Referer header validation
What it CAN do	Execute any authenticated action the victim can perform
What it CANNOT do	Read responses (blind attack) — unless combined with XSS
Defeated by XSS?	Yes — XSS can extract CSRF tokens, bypassing token-based protection
Burp tool	Engagement tools → Generate CSRF PoC